



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Lesson 1: Django Fundamentals

Internet Web & Cloud: Django Basics

Francesco Faenza

Researchers

Division of Cloud Edge and Security

University of Modena and Reggio Emilia

March 9, 2026

What is Django?

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design.

Key Features:

MTV Architecture – Model-Template-View

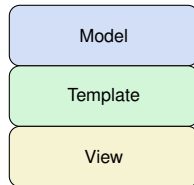
ORM – Object-Relational Mapping

Admin Interface – Auto-generated, powerful

Security – CSRF, XSS, SQL injection protection

Batteries Included – Auth, forms, caching, etc.

Documentation: <https://docs.djangoproject.com/>



The MTV Pattern

MTV Architecture Explained

MTV (Model-Template-View) is Django's architectural pattern – similar to MVC (Model-View-Controller) but with different naming.

Model

Defines the **data structure**. Models are Python classes that describe your database tables and relationships.

```
class Post(models.Model):
```

Handles data validation, relationships, and database operations through the ORM.

Template

Defines the **presentation layer**.

Templates are HTML files with special syntax for displaying dynamic data.

```
{{ post.title }}
```

Separates presentation from business logic. No Python code in templates!

View

Defines the **business logic**. Views process HTTP requests, interact with models, and return responses.

```
def post_list(request):
```

Acts as the glue between models and templates.

Learn more: <https://docs.djangoproject.com/en/stable/faq/general/>

[#django-appears-to-be-a-mvc-framework-but-you-call-the-controller-the-view-and-the-view-t](#)

Why Django?

Pros:

- Rapid development
- Excellent documentation
- Huge community
- Built-in security features
- Scalable (Instagram, Pinterest, Mozilla)
- Django Admin saves hours

Cons:

- Monolithic (batteries included)
- Can be overkill for small projects
- Learning curve for complex features
- Synchronous by default (async support improving)

Django is perfect for data-driven web applications!

Sites using Django: <https://www.djangoproject.com/start/overview/>

Project Setup with UV

UV: Modern Python Package Manager

UV replaces pip + venv + pip-tools – all in one fast tool written in Rust.

Why UV?

Fast – 10-100x faster than pip

Reproducible – Lock file guarantees same versions

Simple – One tool for everything

No activate – `uv run` handles it

UV Documentation: <https://docs.astral.sh/uv/>

Essential Commands:

`uv init` – Create project

`uv add <pkg>` – Add dependency

`uv sync` – Install from lock

`uv run <cmd>` – Run in venv

Creating Your First Django Project

```
# Create project directory
mkdir myproject && cd myproject

# Initialize UV project
uv init

# Add Django
uv add django

# Create Django project
uv run django-admin startproject myproject .

# Run development server
uv run python manage.py runserver
```

The virtual environment is automatically in `.venv/`

Django Tutorial: <https://docs.djangoproject.com/en/stable/intro/tutorial01/>

Django Project Structure

```
myproject/  
|-- manage.py # Django management script  
|-- pyproject.toml # UV dependencies  
|-- uv.lock # Locked versions  
|-- .venv/ # Virtual environment  
'-- myproject/ # Project package  
    |-- __init__.py  
    |-- settings.py # Configuration  
    |-- urls.py # URL routing  
    |-- asgi.py # ASGI entry  
    '-- wsgi.py # WSGI entry
```

Settings Reference: <https://docs.djangoproject.com/en/stable/ref/settings/>

Understanding Django Apps

Django projects are composed of **apps** – reusable, self-contained modules.

Creating an App:

```
uv run python manage.py startapp blog
```

App Structure:

```
blog/  
|-- __init__.py  
|-- admin.py  
|-- apps.py  
|-- models.py # Data layer  
|-- views.py # Business logic  
|-- tests.py  
'-- migrations/
```

Register in settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    ...  
    'blog', # <-- Your app  
]
```

Apps promote reusability and separation of concerns!

Apps docs: <https://docs.djangoproject.com/en/stable/ref/applications/>

Models & ORM

Django Models

Models define your data structure. Django's **ORM** maps Python classes to database tables.

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    published = models.BooleanField(default=False)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return self.title
```

Model Field Reference: <https://docs.djangoproject.com/en/stable/ref/models/fields/>

What is the ORM?

ORM (Object-Relational Mapping) is a technique that lets you interact with your database using Python code instead of SQL.

Without ORM (SQL):

```
SELECT * FROM posts
WHERE published = 1
ORDER BY created_at DESC;

INSERT INTO posts (title, content)
VALUES ('Hello', 'World');
```

With Django ORM:

```
# Query
posts = Post.objects.filter(
    published=True
).order_by('-created_at')

# Create
Post.objects.create(
    title='Hello',
    content='World'
)
```

The ORM handles database compatibility, security, and query optimization!

ORM Documentation: <https://docs.djangoproject.com/en/stable/topics/db/queries/>

Field Types

Common Fields:

CharField – Short strings

TextField – Long text

IntegerField – Integers

FloatField – Decimals

BooleanField – True/False

DateTimeField – Timestamps

EmailField – Validated email

URLField – Validated URL

FileField – File uploads

ImageField – Images

Relationship Fields:

ForeignKey – Many-to-one

OneToOneField – One-to-one

ManyToManyField – Many-to-many

Common Options:

max_length

null=True

blank=True

default=value

unique=True

on_delete=CASCADE

Field Types: <https://docs.djangoproject.com/en/stable/ref/models/fields/#field-types>

Relationships Example

```
from django.db import models
from django.contrib.auth.models import User

class Author(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)

class Category(models.Model):
    name = models.CharField(max_length=100)

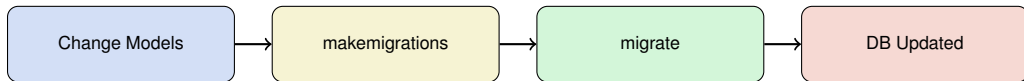
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    categories = models.ManyToManyField(Category)
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

ForeignKey goes on the "many" side of the relationship

Relationships: <https://docs.djangoproject.com/en/stable/topics/db/examples/>

The Migration Workflow -1/2

Migrations are Django's way of propagating model changes to the database schema.



The Migration Workflow -1/2

```
# Step 1: Create migration files (detects model changes)  
uv run python manage.py makemigrations  
# Creates: blog/migrations/0001_initial.py  
  
# Step 2: Apply migrations to database  
uv run python manage.py migrate  
  
# Check migration status  
uv run python manage.py showmigrations  
  
# Rollback if needed  
uv run python manage.py migrate blog zero # rollback all
```

Never edit migration files manually unless you know what you're doing!

Migrations: <https://docs.djangoproject.com/en/stable/topics/migrations/>

Migration Files Explained

Migration files are Python scripts that describe database changes.

```
# blog/migrations/0001_initial.py (auto-generated)
from django.db import migrations, models

class Migration(migrations.Migration):
    initial = True
    dependencies = []

    operations = [
        migrations.CreateModel(
            name='Post',
            fields=[
                ('id', models.AutoField(primary_key=True)),
                ('title', models.CharField(max_length=200)),
                ('content', models.TextField()),
            ],
        ),
    ]
```

Each migration is a version-controlled database change!

Using the ORM

Django's ORM provides a Pythonic way to interact with the database.

Create:

```
post = Post.objects.create(  
    title="Hello",  
    content="World",  
    author=author  
)
```

Read:

```
# All posts  
posts = Post.objects.all()  
# Single post  
post = Post.objects.get(id=1)  
# Filter  
posts = Post.objects.filter(  
    published=True  
)
```

Update:

```
post.title = "New Title"  
post.save()  
  
# Bulk update  
Post.objects.filter(  
    published=False  
) .update(published=True)
```

Delete:

```
post.delete()  
  
# Bulk delete  
Post.objects.filter(  
    created_at__lt=old_date  
) .delete()
```

QuerySet Methods

Filtering:

```
Post.objects.filter(title__icontains='django')
Post.objects.exclude(published=False)
Post.objects.filter(
    Q(title__icontains='django') |
    Q(content__icontains='python')
)
```

Ordering:

```
Post.objects.order_by('-created_at')
Post.objects.order_by('title', '-created_at')
```

Aggregation:

```
Post.objects.count()
Post.objects.filter(published=True).count()
# With annotation
authors = Author.objects.annotate(
    post_count=Count('post')
)
```

Field Lookups:

```
__exact, __iexact
__contains, __icontains
__gt, __gte, __lt, __lte
__in, __startswith
__year, __month, __day
```

QuerySet API: <https://docs.djangoproject.com/en/stable/ref/models/queries/>

Django Admin

The Admin Interface

Django's admin is an auto-generated, production-ready interface for managing data.

Setup:

```
# urls.py
path('admin/', admin.site.urls)

# Create superuser
uv run python manage.py createsuperuser
```

Register Models:

```
from django.contrib import admin
from .models import Post, Author

admin.site.register(Post)
admin.site.register(Author)
```

Custom Admin:

```
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'author', 'published']
    list_filter = ['published', 'created_at']
    search_fields = ['title', 'content']
    date_hierarchy = 'created_at'
    ordering = ['-created_at']
```

Admin Documentation: <https://docs.djangoproject.com/en/stable/ref/contrib/admin/>

Class-Based Views

Why Class-Based Views (CBV)?

Class-Based Views provide an object-oriented way to organize your view logic.

Advantages:

Inheritance – Reuse common patterns

Mixins – Compose functionality

Organization – Methods for each HTTP verb

Built-in generics – ListView, DetailView, etc.

DRY principle – Less code repetition

Comparison:

FBV: One function handles everything

CBV: Methods handle different aspects

`get()` – GET requests

`post()` – POST requests

`get_context_data()` – Add context

`get_queryset()` – Filter data

CBV Documentation: <https://docs.djangoproject.com/en/stable/topics/class-based-views/>

Basic CBV Example 1/2

A simple class-based view that handles GET and POST differently.

```
from django.views import View
from django.shortcuts import render, redirect
from django.http import HttpResponseRedirect

class HelloView(View):
    def get(self, request):
        # Handle GET request
        return render(request, 'hello.html', {'message': 'Hello!'})

    def post(self, request):
        # Handle POST request
        name = request.POST.get('name', 'Guest')
        return render(request, 'hello.html', {'message': f'Hello, {name}!'})
```

Basic CBV Example 2/2

```
# urls.py
from django.urls import path
from .views import HelloView

urlpatterns = [
    path('hello/', HelloView.as_view()),
]
```

Use `.as_view()` to convert the class to a view function!

Generic Display Views 1/2

Django provides powerful generic views for common patterns.

```
from django.views.generic import ListView, DetailView
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = 'blog/post_list.html'
    context_object_name = 'posts'
    paginate_by = 10 # Built-in pagination!

    def get_queryset(self):
        """Filter the queryset."""
        return Post.objects.filter(published=True)

    def get_context_data(self, **kwargs):
        """Add extra context."""
        context = super().get_context_data(**kwargs)
        context['title'] = 'Latest Posts'
        return context
```

Generic Display Views 2/2

```
class PostDetailView(DetailView):  
    model = Post  
    template_name = 'blog/post_detail.html'  
    context_object_name = 'post'
```

Generic views handle 90% of common use cases!

Generic Edit Views 1/2

CRUD operations made simple with class-based views.

```
from django.views.generic import CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy
from .models import Post

class PostCreateView(CreateView):
    model = Post
    fields = ['title', 'content', 'published']
    template_name = 'blog/post_form.html'
    success_url = reverse_lazy('post-list')
```

Generic Edit Views 1/2

```
class PostUpdateView(UpdateView):  
    model = Post  
    fields = ['title', 'content', 'published']  
    template_name = 'blog/post_form.html'  
  
    def get_success_url(self):  
        return reverse_lazy('post-detail', kwargs={'pk': self.object.pk})  
  
class PostDeleteView>DeleteView):  
    model = Post  
    template_name = 'blog/post_confirm_delete.html'  
    success_url = reverse_lazy('post-list')
```

`reverse_lazy` is used because URLs are loaded at runtime

URL Routing for CBVs

```
# blog/urls.py
from django.urls import path
from .views import (
    PostListView, PostDetailView,
    PostCreateView, PostUpdateView, PostDeleteView
)

urlpatterns = [
    path('', PostListView.as_view(), name='post-list'),
    path('post/<int:pk>/', PostDetailView.as_view(), name='post-detail'),
    path('post/create/', PostCreateView.as_view(), name='post-create'),
    path('post/<int:pk>/update/', PostUpdateView.as_view(), name='post-update'),
    path('post/<int:pk>/delete/', PostDeleteView.as_view(), name='post-delete'),
]
```

URL Configuration: Including App URLs

Project-level `urls.py` delegates URL handling to individual apps.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("blog/", include("blog.urls")), # <-- Include blog app URLs
]
```

How it works:

Any request starting with `/blog/` (e.g., `/blog/posts/`) is forwarded to the `blog` app's own URL configuration

The app must have its own `urls.py` file (inside the `blog` folder) that defines the actual routes

Documentation:

<https://docs.djangoproject.com/en/stable/topics/http/urls/#including-other-urlconfs>

CBV' views

Class	What it does
<code>ListView</code>	Display a list of objects
<code>DetailView</code>	Display a single object
<code>CreateView</code>	Create a new object (form)
<code>UpdateView</code>	Update an existing object
<code>DeleteView</code>	Delete an object (confirmation)
<code>TemplateView</code>	Display a static template
<code>RedirectView</code>	Redirect to another URL

Templates

What are Templates?

Templates are text files (usually HTML) that define the presentation layer of your application.

Purpose:

Separate presentation from logic

Reusable HTML structures

Dynamic content rendering

No Python code in HTML

Django Template Language (DTL):

`{{ variable }}` – Print variable

`{% tag %}` – Logic (if, for, etc.)

`{{ value|filter }}` – Transform data

`{# comment #}` – Comments

Template Inheritance:

Templates can extend other templates:

`base.html` – Main layout

`child.html` – Extends base

Template Locations:

`app/templates/app/`

`project/templates/`

Templates: <https://docs.djangoproject.com/en/stable/topics/templates/>

Base Template with Bootstrap

Create a reusable base template using **Bootstrap 5**.

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}My Blog{% endblock %}</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.
    css" rel="stylesheet">
</head>
<body>
```

Base Template (continued)

```
<!-- Navigation -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="{% url 'post-list' %}">My Blog</a>
    <div class="navbar-nav">
      <a class="nav-link" href="{% url 'post-list' %}">Home</a>
      <a class="nav-link" href="{% url 'post-create' %}">New Post</a>
    </div>
  </div>
</nav>
```

Base Template (continued)

```
<!-- Main Content -->
<main class="container my-4">
  {% if messages %}
    {% for message in messages %}
      <div class="alert alert-{{ message.tags }} alert-dismissible fade
        show">
        {{ message }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"></
          button>
      </div>
    {% endfor %}
  {% endif %}

  {% block content %}
  <!-- Child templates override this -->
  {% endblock %}
</main>
```

Base Template (continued)

```
<!-- Bootstrap JS -->  
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.  
    bundle.min.js"></script>  
</body>  
</html>
```

The `{% block %}` tags define areas child templates can override!

Starter template: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Child Template Example

Child templates extend the base and fill in the blocks.

```
<!-- blog/templates/blog/post_list.html -->
{% extends 'base.html' %}

{% block title %}Blog Posts - My Blog{% endblock %}

{% block content %}
  <h1 class="mb-4">Latest Posts</h1>

  <div class="row">
    {% for post in posts %}
      <div class="col-md-6 mb-4">
        <div class="card">
          <div class="card-body">
            <h5 class="card-title">
              <a href="{% url 'post-detail' post.id %}" class="text-decoration-none">
                {{ post.title }}
              </a>
            </h5>
          </div>
        </div>
      </div>
    {% endfor %}
  </div>
{% endblock %}
```

Child Template Example (continue)

```
</h5>
<p class="card-text">{{ post.content|truncatewords:30 }}</p>
<small class="text-muted">
  By {{ post.author }} on {{ post.created_at|date:"F d, Y" }}
</small>
</div>
</div>
</div>
{% empty %}
  <div class="col-12">
    <div class="alert alert-info">No posts available.</div>
  </div>
{% endfor %}
</div>
{% endblock %}
```

Template Tags & Filters

Control Flow:

```
{% if user.is_authenticated %}  
    Welcome, {{ user.username }}!  
{% else %}  
    Please <a href="{% url 'login' %}">log  
        in</a>.  
{% endif %}  
  
{% for item in items %}  
    {{ forloop.counter }}. {{ item }}  
    {% if forloop.first %}(first){% endif %}  
{% empty %}  
    No items found.  
{% endfor %}
```

Template Inheritance:

```
{% extends 'base.html' %}  
{% block content %}...{% endblock %}  
{% include 'header.html' %}
```

Common Filters:

```
var|default:"N/A"  
  
var|length  
  
var|lower , |upper  
  
var|date:"Y-m-d"  
  
var|truncatewords:50  
  
var|safe (be careful!)  
  
var|striptags  
  
var|slugify
```

Built-in filters: <https://docs.djangoproject.com/en/stable/ref/templates/builtins/>

Template Settings

Configure template directories in `settings.py`:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'], # Global templates  
        'APP_DIRS': True, # Look in app templates/ folders  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]
```

Context processors make variables available to all templates!

VSCode Debugging Setup

Debug your Django code directly in VSCode!

```
# .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Django",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/manage.py",
      "args": ["runserver"],
      "django": true,
      "justMyCode": true,
      "python": "${workspaceFolder}/.venv/bin/python"
    }
  ]
}
```

Press F5 to start debugging with breakpoints!

Lesson 1 Summary

What We Learned:

Django MTV architecture

Project setup with UV

Creating Django apps

Models & ORM

Migration workflow

Django Admin

Class-Based Views

URL routing

Templates & DTL

Key Documentation Links:

Django Docs

UV Docs

Model Fields

Migrations

CBVs

Template Filters

Bootstrap Docs

Practice is key!

Questions?

"The best way to learn Django is to build something with it."

– Django Community

Homework: Create a simple blog with CRUD operations using CBVs!